Technical report of the Sheet 3 in Computer Vision II course

Aleksandr Pavlenko
University of Bonn
s97apavl@uni-bonn.de

1. Introduction

This assignment explores convolutional neural network (CNN) architectures for image classification using the CIFAR-10 dataset. The dataset consists of 60,000 color images of size 32×32 across 10 classes, split into 50,000 training and 10,000 test samples.

I began with a simple CNN baseline and progressively modified it through five experimental tasks. These modifications included increasing network depth, adjusting parameter count, introducing regularization and batch normalization, applying data augmentation, incorporating skip connections, and tuning the architecture to meet specific constraints on the number of trainable parameters.

2. Experiments and Results

I conducted a series of experiments corresponding to several architectural and training modifications. Each aimed to improve performance or control complexity while staying within the constraint of approximately 120k parameters. The results include test accuracy, parameter count, and training time. A summary is provided in Table 1 and Figures 1 and 2.

Training Details

All models were trained using stochastic gradient descent (SGD) with a learning rate of 0.02 and momentum of 0.9. Unless otherwise stated, no learning rate scheduling or extensive hyperparameter tuning was applied. Training was conducted in Google Colab using its provided GPU acceleration, which ensured reasonable training times across all experiments.

Task 1: Baseline and Pooling Variants

The baseline model (Task 0) was a shallow CNN without pooling, achieving a test accuracy of 57.15% with around 45k parameters. In Task 1.1, flattening was replaced with global average pooling, slightly lowering accuracy to 54.19%. Task 1.2 removed max-pooling altogether and used strided convolutions, further reducing accuracy to 52.67%. These configurations confirmed that aggressive simplification can hurt performance.

Task 2: Dropout Regularization

In Task 2, dropout was introduced to improve generalization. Although the architecture remained similar to earlier versions, this led to a modest recovery in test accuracy to 54.00%, with roughly 48k parameters and improved training stability.

Task 3: Batch Normalization

Adding batch normalization (Task 3) had a much stronger impact. Accuracy increased to 66.06%, and the model (with ˜60k parameters) trained faster and more stably. This confirmed the expected benefits of normalization layers in small CNNs.

Task 4: Dropout + Batch Normalization

In Task 4, both dropout and batch normalization were applied while keeping parameters below 120k (specifically, 114k). This design achieved 61.92% accuracy, showing a balanced trade-off between regularization and parameter-efficient architecture.

Task 5: Architecture and Data Augmentation

Three models were tested in this phase:

- **Task 5.1 (Data Augmentation):** Augmentations such as random flipping, rotation, and color jitter were applied on top of Task 4. Accuracy improved significantly to 69.78%, with training time increasing to 1140.59 seconds.
- **Task 5.2 (Skip Connections):** Adding residual connections further stabilized training, leading to a test accuracy of 70.53%. The model used 117k parameters and required 1145.60 seconds to train.
- **Task 5.3 (Parametric Expansion + Normalization):** Expanding the model's width and depth, and applying CIFAR-10-specific input normalization, led to the best result of 80.74% accuracy. The model remained within the 120k parameter limit (119k) and trained in 1194.35 seconds.

Table 1. Best Accuracy, Parameter Count, and Training Time per Experiment Variant

ID	Description	Best Accuracy (%)	# Params	Train Time (s)
Task 0	Base CNN (no pooling, no tricks)	57.15	˜45k	˜259
Task 1.1	Global Average Pooling instead of Flatten	54.19	˜45k	˜258
Task 1.2	No MaxPooling, Strided Convolutions Only	52.67	˜45k	˜244
Task 2	+ Dropout (Regularization)	54.00	˜48k	˜248
Task 3	+ Batch Normalization	66.06	˜60k	˜267
Task 4	+ Dropout + Batch Normalization	61.92	˜114k	˜271
Task 5.1	model from task 4 + Data Augmentation Only	69.78	˜115k	˜1140
Task 5.2	model from task 5.1 + Skip Connections	70.53	˜117k	˜1145
Task 5.3	model from task 5.2 + Extra Parameters + CIFAR-10 Normalization (Best)	**80.74**	119k	˜1194

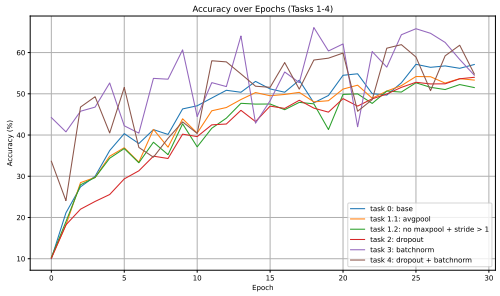


Figure 1. Accuracy over epochs for Tasks 1–4

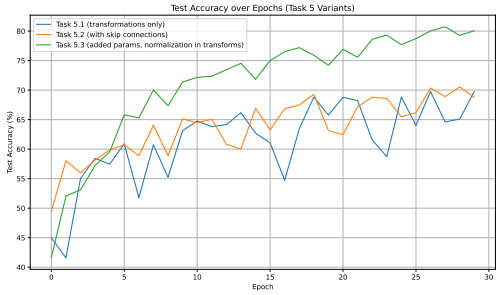


Figure 2. Accuracy over epochs for 3 models in Task 5

3. Discussion

Performance Trends

The results demonstrate that even relatively simple CNNs benefit significantly from progressive architectural tuning and training refinements.

Pooling Variants (Tasks 1.1–1.2): Replacing flattening with global average pooling (Task 1.1) and eliminating max pooling in favor of strided convolutions (Task 1.2) slightly degraded performance. This suggests that while these techniques can reduce complexity, they may sacrifice critical spatial information when applied in isolation.

Dropout Regularization (Task 2): The addition of dropout modestly improved generalization, recovering part of the lost accuracy. However, the benefit was limited due to the overall small capacity of the network.

Batch Normalization (Task 3): Introducing batch normalization led to a substantial accuracy boost (from 54.00% to 66.06%). It also improved training stability and convergence speed, confirming its role as a key enabler for deeper learning even in small models.

Combined Regularization (Task 4): The combination of dropout and batch normalization in a model constrained to 114k parameters yielded 61.92% accuracy. Although slightly lower than Task 3, this design respected the parameter budget and provided a robust base for further improvements.

Data Augmentation and Skip Connections (Task 5)

Task 5.1 – Augmentations: Applying basic transformations such as flipping, rotation, and color jitter significantly improved performance to 69.78%. The increase in input diversity contributed to better generalization, despite no architectural changes.

Task 5.2 – Skip Connections: Incorporating residual connections yielded a further improvement to 70.53%. This suggests that improved gradient flow helps deeper representations even in relatively shallow networks.

Task 5.3 – Parametric Expansion + Normalization: This was the most impactful intervention. Expanding the model's depth and width and normalizing input data using CIFAR-10 statistics pushed accuracy to 80.74%. This confirms that both architectural capacity and dataset-specific

preprocessing are essential for high performance within parameter limits.

Efficiency

While training time increased with model complexity—ranging from around 250 seconds to nearly 1200—the runtime remained under 20 minutes in all cases. Even for the best-performing model (Task 5.3), training was efficient, and inference time remained low, making it suitable for resource-constrained applications.

Failure Cases and Limitations

Some augmentation schemes (e.g., in Task 5.1) caused instability in early training stages. Without proper normalization (as seen in Tasks 5.1 and 5.2), performance plateaued below 71%, indicating the importance of tailored preprocessing.

4. Conclusion

In this report, I analyzed the impact of progressive modifications to convolutional neural network architectures for image classification on the CIFAR-10 dataset. Starting from a basic CNN without pooling or regularization, I evaluated a range of changes including pooling strategies, regularization, normalization, architectural expansion, and data augmentation, all under a strict parameter budget of approximately 120k.